

INTERNSHIP PROJECT REPORT

Project Title: Bookstore REST API

Student Name:	Vivek C Raj	Internship Organization:	Elevate Labs
Education:	BCA	Project Type:	Internship Project
Project Duration:	2 Weeks	Domain:	Backend Engineering

1. Introduction

The Bookstore REST API is a lightweight, high-performance backend application developed during the 2-week Internship Project Phase at **Elevate Labs**. Designed to facilitate efficient inventory management for bookstores, this project provides a robust suite of RESTful endpoints enabling client applications to perform full CRUD (Create, Read, Update, Delete) operations and keyword-based searches on book inventory.

In modern web application development, REST APIs serve as the core communication layer between frontend interfaces and persistent storage. This project addresses fundamental inventory management requirements by maintaining structured database records for each book, including database ID, title, author, price, and quantity. The application leverages a relational SQLite database managed via Object-Relational Mapping (ORM) to ensure reliable data persistence, automated request payload validation, and standardized HTTP status codes. Interactive documentation via Swagger UI and end-to-end API testing using Postman were integrated into the development workflow to verify endpoint execution and system reliability.

2. Abstract

This report details the design and implementation of the Bookstore REST API developed at **Elevate Labs** using Python and the FastAPI framework. FastAPI was selected for its high execution speed, automatic OpenAPI documentation generation, and native integration with modern asynchronous Python standards and Pydantic data schemas.

The application implements key RESTful endpoints to manage bookstore items: creating new book records, fetching all catalog items, retrieving detailed records by unique ID, updating existing book parameters, deleting records, and searching books by title or author keywords. Data consistency and type safety are enforced using Pydantic schemas, which validate incoming JSON payloads prior to database operations. SQLAlchemy is employed as the Object-Relational Mapper (ORM) to interface with the SQLite database seamlessly. Comprehensive error handling is built-in, returning standard 404 HTTP status codes whenever a requested resource is non-existent. Version control and code repository hosting were managed using Git and GitHub throughout the 2-week duration.

3. Tools Used

Category / Tool	Technology	Purpose / Usage in Project
Programming Language	Python	Core programming language for application backend logic.
Web Framework	FastAPI	Modern web framework used to construct RESTful API endpoints.
ASGI Server	Uvicorn	Asynchronous server interface for running the application.
Database ORM	SQLAlchemy	Object-Relational Mapper for SQLite database interactions.
Database	SQLite	Relational database engine for bookstore inventory storage.
Data Validation	Pydantic	Request data parsing, validation, and schema definitions.
API Documentation	Swagger UI	Interactive, autogenerated API documentation and testing interface.
API Testing	Postman	External client testing for endpoints and HTTP status verification.
Version Control	Git & GitHub	Source code version management and remote repository hosting.

4. Steps Involved in Building the Project

1. **Environment Setup:** Set up the Python project directory and virtual environment (venv).
2. **Dependency Installation:** Installed required Python dependencies (fastapi, uvicorn, sqlalchemy, pydantic).
3. **FastAPI Application Setup:** Created the core FastAPI application instance and routing logic.
4. **Database Configuration:** Configured the SQLite database connection and session engine using SQLAlchemy.
5. **Book Model Definition:** Created the Book database model with fields: id, title, author, price, and quantity.
6. **Validation Schemas Creation:** Created Pydantic schemas for request payload validation and response serializations.
7. **CRUD API Implementation:** Implemented RESTful CRUD endpoints for creating, reading, updating, and deleting books.
8. **Search Functionality:** Implemented book search functionality filtering records by title or author keywords.
9. **Validation & Error Handling:** Added input validation and 404 HTTP error handling for non-existent book records.
10. **Swagger UI Testing:** Tested the API interactive endpoints using auto-generated Swagger UI (/docs).
11. **Postman Verification:** Tested and verified API endpoints and status codes using Postman.
12. **Version Control:** Created a Git repository and uploaded the complete codebase to GitHub.

5. Conclusion

The Bookstore REST API project was successfully built and tested during the 2-week Internship Project Phase at **Elevate Labs**. The project fulfills all functional requirements for bookstore inventory management, demonstrating effective implementation of Python backend development principles, RESTful API architecture, CRUD operations, and relational database management.

By completing this internship project, practical competencies were demonstrated in building robust web services using FastAPI, SQLAlchemy, SQLite, and Pydantic. Rigorous validation and testing workflows using Swagger UI and Postman ensured high software reliability and standards.

The final codebase remains cleanly organized and hosted on GitHub.